ROADRESQ

Security Architecture & Threat Model

Version 1.0 • Production-Oriented Security Baseline

Scope: Customer • Provider • Admin • API • Realtime • Payments • Location • Data
Security posture: Zero-trust boundaries + least privilege + defense in depth

IDENTITY → AUTHORIZATION → DATA → REALTIME → PAYMENTS → AUDIT → RESPONSE

Document Control

Field	Value
Document	Security Architecture & Threat Model
Product	RoadResQ
Version	1.0
Status	Production security baseline
Audience	Engineering, security, DevOps, QA, operations and product
Primary stack	Next.js / React • FastAPI • PostgreSQL/PostGIS • Redis • WebSockets

1. Security Objectives

• Protect customer, provider, administrative, location and financial information.

• Prevent unauthorized access to accounts, vehicles, bookings, documents, payments and operational data.

• Preserve integrity of the booking state machine, provider assignment, estimates and financial records.

• Keep realtime location and communication accessible only to authorized participants.

• Reduce fraud, abuse, credential attacks, data leakage and service disruption.

• Provide sufficient auditability to investigate security and operational incidents.

• Design for secure recovery when third-party services or infrastructure fail.

2. Security Principles

• **Least privilege:** every user, service and database role receives only necessary permissions.

• **Zero trust:** authentication does not automatically imply authorization to every object.

• **Defense in depth:** edge, API, application, database and storage controls complement each other.

• **Server authority:** clients cannot directly decide booking state, payment success or privileged outcomes.

• **Secure by default:** private storage, restrictive CORS, safe headers and disabled unnecessary capabilities.

• **Fail securely:** uncertain authentication/payment/authorization states do not default to success.

• **Privacy by design:** collect and retain only information required for the service.

3. System Trust Architecture

Logical trust flow:

```
PUBLIC CLIENTS → CDN/WAF → LOAD BALANCER → API → DOMAIN SERVICES → DATABASE / REDIS / OBJECT STORAGE → EXTERNAL PROVIDERS
```

Zone	Trust assumption	Primary controls
Public clients	Untrusted	TLS, validation, rate limits, auth, secure client storage
Edge	Controlled boundary	WAF/rate limiting, request limits, TLS termination
Application	Trusted execution but attackable	RBAC, object authorization, validation, secure config
Database	Highly restricted	Private network, least privilege, encryption, backups
Redis	Restricted transient system	Network isolation, auth, TTLs, no authoritative financial state
Object storage	Restricted data store	Private buckets, signed URLs, validation, retention

Zone	Trust assumption	Primary controls
Admin plane	Highly privileged	Strong auth, narrow roles, audit logs, session controls
Third parties	Untrusted dependency	Adapter isolation, timeouts, signatures, retries, reconciliation

4. Identity & Authentication Architecture

4.1 Authentication Requirements

• Use verified phone/email identity as configured by the product.

• OTP requests and verification must be rate-limited and protected against brute force.

• Access credentials must have bounded lifetime and secure refresh/revocation behavior.

• Logout/session revocation must invalidate the relevant credential/session.

• Account status must be checked during authorization for sensitive operations.

• Administrative access should use stronger authentication and shorter session controls than ordinary customer flows.

4.2 Credential Protection

• Never log passwords, OTP values, refresh tokens, access tokens or payment secrets.

• Use modern password hashing if password authentication is enabled.

• Store refresh/session credentials securely and rotate/revoke where appropriate.

• Do not expose internal authentication implementation details in errors.

5. Authorization Architecture

RoadResQ requires both role-based access control (RBAC) and object-level authorization.

Role	Allowed high-level access
CUSTOMER	Own profile, own vehicles, eligible own bookings, own payments/history, own reviews
PROVIDER	Own provider profile/documents, eligible requests, assigned jobs, own earnings
SUPPORT	Assigned support/ticket operations and narrowly scoped customer/job information
ADMIN	Privileged platform operations according to explicit admin permissions

Object authorization examples: a customer requesting **/vehicles/{id}** must own that vehicle; a provider requesting a booking must be assigned/authorized for that job; support users must only see records permitted by support policy.

6. API Security Architecture

• Use HTTPS/TLS for all production API traffic.

• Validate every request with typed schemas and business-rule validation.

• Apply authentication middleware before protected handlers.

• Apply object-level authorization inside service boundaries rather than relying only on UI restrictions.

• Use allow-listed sorting/filtering fields to reduce injection/abuse risks.

• Use bounded pagination and request-body limits.

• Apply rate limits to authentication, OTP, booking creation, location updates and other abuse-sensitive endpoints.

• Use idempotency keys for payment and other retryable critical writes.

• Return generic errors for security-sensitive authentication failures.

• Generate request IDs for traceability.

6.1 CORS / Headers / Browser Security

• Allow only approved frontend origins in production.

• Use secure response headers appropriate to the web application.

• Configure cookies with Secure, HttpOnly and appropriate SameSite attributes if cookies are used.

• Use CSRF protection where cookie-based authentication creates CSRF exposure.

• Do not permit arbitrary framing or unsafe inline behavior without an explicit requirement.

7. Database Security Architecture

• Keep PostgreSQL private and inaccessible directly from public clients.

• Use separate database credentials/roles for application and administrative functions.

• Restrict database privileges according to service responsibility.

• Enforce foreign keys, unique constraints and check constraints for security-relevant integrity.

• Use parameterized queries/ORM mechanisms; never construct SQL from untrusted strings.

• Encrypt backups and restrict restore access.

• Monitor unusual connection counts, failed authentication and expensive/suspicious queries.

• Do not store raw passwords, OTPs or unnecessary payment credentials.

8. Redis Security Architecture

• Keep Redis on a private network.

• Require authentication and appropriate ACLs where supported.

• Use TTLs for OTPs, locks, temporary tokens and ephemeral state.

• Never treat Redis as the authoritative source for payments or irreversible booking outcomes.

• Protect pub/sub channels from cross-tenant or cross-booking leakage.

• Limit memory growth and configure eviction/operational safeguards appropriately.

9. Object Storage & Upload Security

• Store provider verification documents and media in private buckets/containers.

• Accept only allow-listed file types and bounded file sizes.

• Generate non-guessable object keys.

• Scan uploaded content for malware where operationally appropriate.

• Use signed URLs with short expiry for authorized retrieval.

• Do not expose raw storage credentials to clients.

• Maintain retention/deletion rules for documents and media.

• Never trust a filename or client-provided MIME type as the sole security control.

10. Realtime & Location Security

10.1 WebSocket Security

• Require authentication during WebSocket connection establishment.

• Authorize the user for the requested booking/channel before subscribing.

• Limit subscriptions to eligible resources.

• Validate all client-sent realtime messages.

• Disconnect or re-authenticate expired/invalid sessions.

• Rate-limit high-frequency events and enforce message-size limits.

• Reconnect logic must re-fetch authoritative state through the API.

10.2 Location Privacy

• Collect precise location only when required for active assistance/tracking.

• Do not expose provider location to unrelated users.

• Stop or reduce tracking after the operational context ends.

• Use retention policies and sampling rather than indefinite raw GPS storage.

• Restrict location access in admin/support interfaces to legitimate operational needs.

• Audit access to sensitive location data where practical.

11. Payment Security Architecture

• Use a trusted payment gateway for payment processing rather than storing raw card credentials.

• Verify signed payment webhooks.

• Treat gateway-confirmed server status as authoritative, not a client success screen.

• Use idempotency keys to prevent duplicate payment creation.

• Handle delayed/unknown gateway status through reconciliation.

• Separate customer payment records from provider payout records.

• Restrict financial endpoints and audit privileged changes.

• Protect gateway keys in secure secret management and rotate them when required.

Payment security flow:

```
CLIENT → PAYMENT API → GATEWAY → SIGNED WEBHOOK → VERIFY → IDEMPOTENT UPDATE → PAYMENT RECORD → RECONCILIATION
```

12. Threat Modeling Method

Threats are grouped using STRIDE-style categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege. Risk is prioritized using likelihood and impact rather than relying only on technical severity.

Risk scale: **Critical / High / Medium / Low**. Final ratings should be revalidated after deployment topology and actual integrations are selected.

13. Threat Register

ID	Threat	Impact	Primary mitigation	Risk
TM-01	Account takeover through stolen/abused OTP	Unauthorized account access	OTP rate limits, expiry, attempt limits, anomaly detection, secure session management	High
TM-02	Broken object-level authorization	Cross-user vehicle/booking/data access	Server-side ownership/relationship checks + authorization tests	Critical

ID	Threat	Impact	Primary mitigation	Risk
TM-03	Provider impersonation	Fraudulent roadside service	Verification workflow, status controls, audit trail, provider identity checks	High
TM-04	Race to accept same booking	Wrong provider assignment	DB transaction/unique constraint + concurrency tests	High
TM-05	Client forges booking status	Fraudulent completion/payment flow	Server-authoritative state machine	Critical
TM-06	Payment webhook forgery	False payment success	Signature verification + idempotency + reconciliation	Critical
TM-07	Duplicate payment submission	Double charge/record	Idempotency keys + gateway reconciliation	High
TM-08	Location leakage	Privacy/safety harm	Booking-scoped authorization, retention limits, encrypted transport	High
TM-09	Malicious file upload	Malware/storage abuse	Allow-list, size limits, scanning, private storage	High
TM-10	API brute force/abuse	Account or service compromise	Rate limiting, lockout/backoff, monitoring	High
TM-11	SQL/command injection	Data/system compromise	Parameterized queries, validation, least privilege	Critical
TM-12	XSS in chat/reviews	Session/data compromise	Output encoding, sanitization, CSP strategy	High
TM-13	WebSocket subscription abuse	Cross-booking data leak	Authenticate + authorize every channel	High
TM-14	DDoS/resource exhaustion	Service outage	WAF/rate limits, quotas, autoscaling and capacity controls	High
TM-15	Secrets committed to source	Credential compromise	Secret scanning, CI policy, secret manager, rotation	Critical
TM-16	Admin account compromise	Platform-wide control	Strong authentication, least privilege, audit, session controls	Critical
TM-17	Sensitive logs	PII/secret leakage	Structured logging policy + redaction + access control	High
TM-18	Redis compromise	Transient data/control abuse	Private network, ACLs, TTLs, least privilege	High
TM-19	Third-party API outage	Degraded service	Timeouts, retries, fallback states, adapters	Medium
TM-20	Database deletion/corruption	Data loss/outage	Backups, PITR where available, restore drills, migration controls	Critical

14. Abuse & Fraud Scenarios

• Customer repeatedly creates fake assistance requests to consume provider capacity.

• Provider accepts requests and repeatedly cancels to manipulate dispatch.

• Multiple accounts are created to abuse promotions or ratings.

• User attempts to access another booking by guessing identifiers.

• Fake or manipulated provider documents are submitted.

• Customer/provider attempts to bypass estimate approval through client manipulation.

• Payment callbacks are replayed or sent out of order.

• Review endpoints are spammed or duplicated.

• Location updates are spoofed to manipulate ETA or matching.

Mitigations should combine rate limits, business-rule checks, anomaly signals, audit trails, verification, idempotency and operational review rather than relying on a single control.

15. Security Logging & Audit

Event class	Examples	Requirement
Authentication	Login, OTP verification, logout, failed attempts	Log security-relevant outcome; never log secrets
Authorization	Denied resource access	Record safe actor/resource context
Admin	Provider verification, suspension, user restrictions	Immutable/auditable record
Booking	Assignment, cancellation, status transitions	Persist history with actor/time
Financial	Invoice finalization, payment state, payout changes	Strong audit and reconciliation
Data access	Sensitive document/location access where appropriate	Controlled logging
Security	Rate-limit triggers, suspicious activity	Alertable security telemetry

16. Incident Response Architecture

16.1 Incident Lifecycle

```
DETECT → TRIAGE → CONTAIN → ERADICATE → RECOVER → VALIDATE → POST-INCIDENT REVIEW
```

• Maintain severity levels and named incident ownership.

• Immediately revoke/rotate compromised credentials or secrets.

• Contain affected accounts, providers, endpoints or integrations.

• Preserve relevant logs and audit evidence.

• Restore from trusted backups when data integrity is affected.

• Validate security controls before reopening affected functionality.

• Document root cause, timeline, impact and preventive actions.

16.2 Example Incidents

Incident	Immediate actions
Stolen API secret	Disable/rotate secret, inspect usage, identify affected systems, redeploy safely.
Admin compromise	Revoke sessions, disable account, reset credentials, review audit logs, assess changes.

Incident	Immediate actions
Payment fraud	Pause affected flow if needed, verify gateway records, reconcile transactions, investigate.
Data exposure	Contain access, identify scope, preserve evidence, apply access fix and follow applicable response obligations.
Database corruption	Stop destructive writers if needed, assess integrity, restore/recover, validate application consistency.

17. Security Testing Strategy

• Unit tests for authorization/business rules.

• API integration tests for RBAC and object ownership.

• Negative tests for every protected endpoint.

• Automated dependency and secret scanning in CI.

• Static analysis and linting for backend/frontend.

• Upload security tests.

• Rate-limit and abuse tests.

• WebSocket authorization/reconnect tests.

• Payment webhook signature/replay/idempotency tests.

• Concurrency tests for booking acceptance and critical state transitions.

• Periodic penetration testing before major production milestones.

18. Secure SDLC

Stage	Security activity
Design	Threat model, trust boundaries, abuse cases, privacy review
Development	Secure coding standards, typed validation, dependency hygiene
Pull request	Code review, static checks, tests, secret scanning
CI	Dependency/SAST checks, unit/integration/security tests
Staging	E2E, authorization, upload, payment and failure-path testing
Release	Change approval, migration review, rollback readiness
Production	Monitoring, alerting, audit review, vulnerability management
Post-release	Incident review, patching, threat-model updates

19. Security Configuration Baseline

• Production debug mode disabled.

• Secrets supplied through secure configuration/secret management.

• Database and Redis private-network access only.

• Strict production CORS allow-list.

• TLS enforced.

• Secure headers configured.

• Rate limits enabled.

• File upload limits enabled.

• Admin endpoints protected by privileged authorization.

• Payment webhook verification enabled.

• Audit logging enabled for privileged actions.

• Backups encrypted and restore-tested.

• Dependency vulnerabilities monitored and remediated.

20. Security Requirements by Component

Component	Minimum security controls
Frontend	Safe rendering, secure token/cookie handling, CSP strategy, no secrets in client bundle
FastAPI	Authentication middleware, RBAC, object authorization, validation, rate limits, secure errors
PostgreSQL/PostGIS	Private access, least privilege, constraints, encryption/backups, query safety
Redis	Private access, authentication/ACLs, TTLs, bounded usage
WebSockets	Authenticated connection, channel authorization, rate/message limits
Object storage	Private objects, signed URLs, upload validation, retention
Payment gateway	Server-side integration, signed webhooks, idempotency, reconciliation
Maps	API key restrictions, server/client separation as appropriate, quota controls
Workers	Least-privilege credentials, idempotent jobs, queue controls
Admin portal	Strong authentication, narrow roles, audit logs, session controls

21. Security Acceptance Criteria

• No customer can access another customer's vehicle, booking, invoice, message or review through identifier manipulation.

• No provider can access unrelated provider/customer operational data.

• Invalid booking state transitions are rejected server-side.

• Concurrent provider acceptance results in a single valid assignment.

• Payment webhook signatures are verified and duplicate events are harmless.

• Sensitive credentials and OTPs are absent from logs.

• Unauthorized WebSocket subscriptions are rejected.

• Uploads cannot bypass file/type/size/security controls.

• Rate limits block repeated abuse-sensitive requests.

• Admin actions are auditable.

• Backup restoration has been successfully tested.

• Critical/high security findings are resolved or explicitly accepted before production.

22. Security Roadmap

Phase	Security focus
MVP	Auth, RBAC, object authorization, TLS, rate limits, secure uploads, payment webhook verification, audit logs

Phase	Security focus
Pre-production	Threat-model review, automated security tests, dependency scanning, penetration testing, backup restore drill
Scale	WAF/DDoS controls, centralized secrets, advanced anomaly detection, stronger observability
Advanced	Fraud scoring, device/risk signals, security analytics, formal third-party assessments where justified

23. Final Security Summary

RoadResQ should treat identity, authorization, location, financial integrity and operational state as high-value security boundaries. The most important controls are server-side object authorization, a server-authoritative booking state machine, secure provider verification, authenticated/authorized realtime channels, payment webhook verification with idempotency and reconciliation, private document storage, rate limiting, least-privilege infrastructure and strong auditability. Threat modeling is a living process and should be updated whenever major features, integrations or deployment architecture change.